

Jegyzőkönyv

Web Technológiák 2. Féléves feladat

Kovács Bt. – Munkaóra Nyilvántartó Webalkalmazás

Készítette: Molnár Márk

Neptun-kód: TLZ12Y

Miskolc, 2026

Tartalomjegyzék

1. Bevezetés.....	3
2. Feladat leírása.....	3
3. Rendszerarchitektúra és felépítés.....	3
3.1. Backend (Node.js, Express, MongoDB).....	3
3.2. Frontend (Angular).....	4
4. Felhasználói szerepkörök és funkciók.....	4
4.1. Dolgozói felület (Dashboard).....	4
4.2. Adminisztrátori felület.....	6
5. Kiemelt technikai megoldások (Frontend).....	8
5.1. Komponens alapú felépítés.....	8
5.2. Dinamikus Sötét mód (Dark Theme).....	9
5.3. Űrlapellenőrzés és Regex validáció.....	9
6. Kiemelt technikai megoldások (Backend & Adatbázis).....	9
6.1. REST API és végpontok.....	9
6.2. Dátumok és időzónák kezelése.....	10
6.3. DNS feloldás kényszerítése (MongoDB Atlas kapcsolat).....	10
7. Összegzés, tapasztalatok.....	10

1. Bevezetés

A modern webes alkalmazások fejlesztése ma már elképzelhetetlen a kliens- és szerveroldal éles szétválasztása, valamint a megfelelő keretrendszerek használata nélkül. A Web technológiák 2. féléves projektjének célja egy teljes (Full-Stack) webalkalmazás elkészítése volt, modern JavaScript alapú technológiákkal.

A választott témám egy munkaóra-nyilvántartó rendszer, amelyet a fiktív "Kovács Bt." nevű logisztikai vállalat számára készítettem. Az alkalmazás megoldást nyújt a dolgozók napi munkaidejének precíz rögzítésére, kategóriákba sorolására (pl. Szállítás, Könyvelés), valamint az adminisztrátorok számára egy áttekinthető, statisztikákkal és üzenetküldési lehetőségekkel ellátott vezérlőpultot biztosít.

2. Feladat leírása

A tantárgyi követelmények alapján egy kliens-szerver architektúrájú alkalmazást kellett létrehozni. A projektnek tartalmaznia kellett:

- Backend szerver REST API-val (Node.js + Express).
- Adatbázis-kapcsolatot és adatok perzisztens tárolását (MongoDB + Mongoose).
- Egy komponens-alapú frontend keretrendszer (Angular) használatát.
- Több nézetet, útválasztást (Routing).
- Felhasználói regisztrációt és bejelentkezést.
- Dinamikus DOM-manipulációt és űrlapkezelést.

Az általam készített rendszer mindezeket a funkciókat magában foglalja, kiegészítve jogosultságkezeléssel (admin, dolgozó), személyre szabott kommunikációval, és egy reszponzív, Angular Material alapú felhasználói felülettel.

3. Rendszerarchitektúra és felépítés

A projekt fájlstruktúrája két fő mappára oszlik, élesen elválasztva a szerver és a kliens kódját.

3.1. Backend (Node.js, Express, MongoDB)

A szerveroldal feladata az adatok tárolása, a biztonságos kiszolgálás és a bonyolultabb statisztikai számítások elvégzése.

Keretrendszer: Express.js, amely a RESTful API végpontokat (GET, POST, PUT, DELETE) biztosítja.

Adatbázis: MongoDB, Mongoose ODM (Object Data Modeling) sémákkal (User és Munkaóra).

Modellek:

- User: Tárolja a felhasználók nevét, e-mail címét, jelszavát, szerepkörét (admin/dolgozó) és az admin által küldött személyes üzenetet.
- Munkaóra: Tárolja a dolgozó azonosítóját, a dátumot, a feladat kategóriáját, a kezdés/befejezés idejét, és a kiszámolt óraszámot.

3.2. Frontend (Angular)

A kliensoldal felel a felhasználói élményért (UI/UX) és az adatok megjelenítéséért.

Keretrendszer: Angular 17+ (Standalone Components architektúrával).

UI Könyvtár: Angular Material (kártyák, űrlapmezők, dialógusablakok, chipek).

Szolgáltatások (Services): AuthService és AdminService az API-val való kommunikációra.

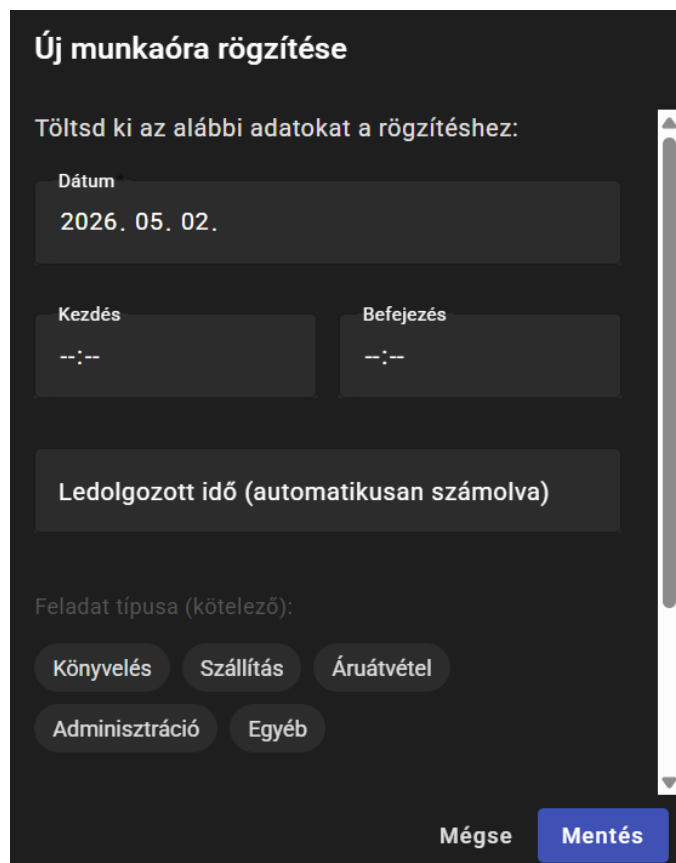
Stílus: SCSS, támogatva az egyedi sötét/világos témákat.

4. Felhasználói szerepkörök és funkciók

4.1. Dolgozói felület (Dashboard)

A normál jogosultsággal belépő felhasználó egy letisztult felületet lát, ahol csak a saját adatait kezelheti.

Új munkaóra rögzítése: Egy felugró (Dialog) ablakban adható meg a dátum, a kezdés és a befejezés ideje. A rendszer automatikusan, tizedespontossággal kiszámolja a ledolgozott időt. Kötelező kiválasztani egy tevékenységi kategóriát is.



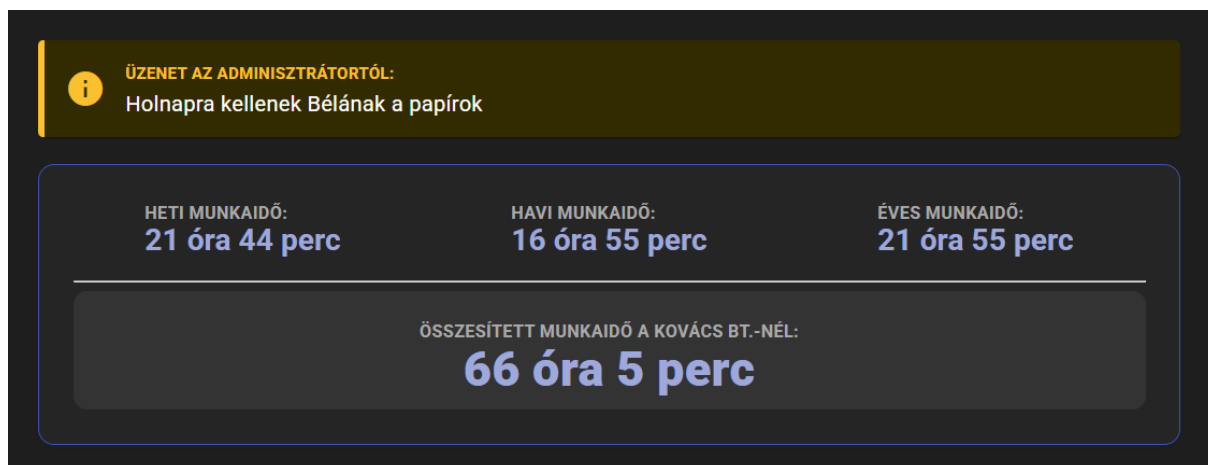
1. ábra Új munkaóra rögzítése

```
emailGeneralas() {
  const adatok = this.getHetiOsszefoglaloAdatai();
  const mailtoLink =
`mailto:${adatok.cimzett}?subject=${encodeURIComponent(adatok.targy
)}&body=${encodeURIComponent(adatok.torzs)}`;
  window.location.href = mailtoLink;
}
```

1. kód Email küldés

Napi/Heti nézet: A felvitt adatok listázva jelennek meg, lehetőség van az elrontott bejegyzések törlésére és javítására.

Admin üzenet panel: A képernyő tetején megjelenik a vezetőség által küldött személyes üzenet, így biztosítva az aszinkron kommunikációt.



2. ábra Üzenet és összesített munkaórák

4.2. Adminisztrátori felület

A szerepkör: 'admin' attribútummal rendelkező felhasználó egy kiterjesztett vezérlőpultot kap.

Statisztikai nézet: Listázza az összes dolgozót (az adminok kivételével). Kártyás nézetben látható a dolgozók adott heti, illetve az eddigi összes ledolgozott órája.

Részletes heti napló: A kártyákon egy görgethető lista mutatja a dolgozó adott heti összes bejegyzését, napra és kategóriára bontva.

Dinamikus időutazás: Beépített hétváltó gombokkal (-7 nap, +7 nap, "Ma") az admin vissza tud keresni korábbi időszakokat is. A statisztikák a kiválasztott hétfő–vasárnap intervallumnak megfelelően frissülnek.

Üzenetküldés és dolgozókezelés: Az admin tud új dolgozót regisztrálni a rendszerbe, illetve törölni a meglévőket. Továbbá minden dolgozó kártyáján található egy szövegmező személyes üzenetek rögzítéséhez.

Új dolgozó regisztrálása

Dolgozó neve

E-mail cím

Jelszó

Jelszó megerősítése

FIÓK LÉTREHOZÁSA

3. ábra Új dolgozó regisztrálása

Könyvelő Klára
konyvelo@kovacsbt.hu

HETI TELJESÍTMÉNY: 0 perc

E HETI RÖGZÍTÉSEK:

Nincs rögzített munka ezen a héten.

Üzenet a dolgozónak

➤ Üzenet mentése

Mindösszesen rögzítve:

8 óra

🗑️ Fiók törlése

4. ábra Egy dolgozó adatlapja - admin nézet

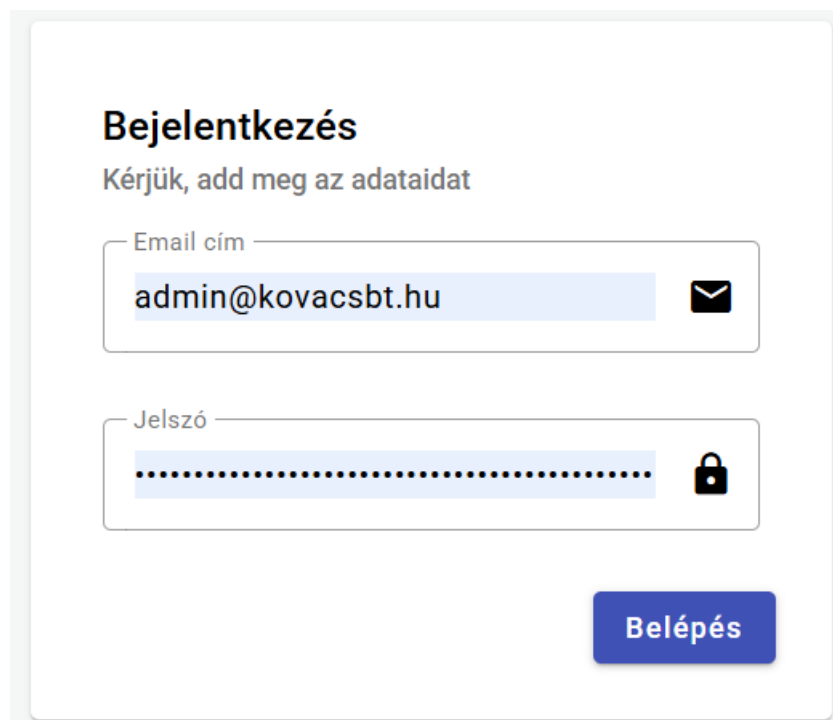
```
router.delete('/felhasznalo/:id', async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id);
    await Munkaora.deleteMany({ felhasznaloId: req.params.id });
    res.status(200).json({ üzenet: 'Törölve!' });
  } catch (error) {
    res.status(500).json({ hiba: 'Hiba a törlés során.' });
  }
});
```

2. kód Dolgozó törlése

5. Kiemelt technikai megoldások (Frontend)

5.1. Komponens alapú felépítés és Change Detection

Az alkalmazás logikai egységekre (Login, Dashboard, Admin) van bontva. Amikor az admin egy új dolgozót regisztrál, vagy módosít egy üzenetet, az Angular ChangeDetectorRef szolgáltatása segítségével a felület azonnal (oldalfrissítés nélkül) újrarajzolja a statisztikai adatokat.



5. ábra Bejelentkezés

```
this.adminService.getStatisztikak(datumString).subscribe({
  next: (valasz) => {
    this.dolgozokStatisztikakkal = valasz.adatok;
    this.cdr.detectChanges();
  },
  error: (err) => alert('Hiba a dolgozók betöltésekor!')
});
```

3. kód detectChanges

5.2. Dinamikus Sötét mód (Dark Theme)

Az alkalmazás beépített sötét móddal rendelkezik, amely nem csak a hátteret, hanem az Angular Material elemek (Dialogusok, input mezők) stílusát is globálisan (styles.scss) felülírja. A választott témát a rendszer a böngésző localStorage-jában tárolja, így az oldal újratöltésekor is megmarad.

5.3. Űrlapellenőrzés és Regex validáció

A regisztrációs űrlapon kiemelt figyelmet kapott a biztonság. A rendszer reguláris kifejezésekkel (RegEx) ellenőrzi az adatokat a küldés előtt:

E-mail: Ellenőrzi a megfelelő formátumot (@ jel és érvényes domain végződés jelenléte).

Jelszó: Szigorú követelmények kikényszerítése: legalább 6 karakter, kis- és nagybetű, minimum egy szám és egy speciális karakter.

```
const jelszoMintazat = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&.])([A-Za-z\d@$!%*?&.]){6,}$/;

if (!jelszoMintazat.test(this.ujDolgozo.jelszo)) {
  alert('✗ Túl gyenge jelszó!\n\nElvárások:\n- Legalább 6 karakter\n- Kis- és nagybetű\n- Legalább egy szám\n- Legalább egy speciális karakter (@$!%*?&. )');
  return;
}

if (this.ujDolgozo.jelszo !== this.ujDolgozo.jelszoUjra) {
  alert('✗ A két jelszó nem egyezik!');
  return;
}
```

4. kód Jelszó ellenőrzés

6. Kiemelt technikai megoldások (Backend & Adatbázis)

6.1. REST API és végpontok

A backend API szeparált routerekkel dolgozik. Az admin útvonal összetett adatbázis-lekérdezéseket végez. Egy kéréssel behúzza a dolgozók listáját és az összes munkaórát, majd a memóriában (JavaScript array filter és reduce függvényekkel) összesíti a heti és az élettartam-órákat minden egyes dolgozóhoz, csökkentve ezzel a hálózati terhelést.

6.2. Dátumok és időzónák kezelése

A munkaórák statisztikáinak lekérésekor a rendszer intelligensen kezeli a hetek határait. A frontend által küldött helyi (YYYY-MM-DD) dátumot a backend alapul véve kiszámolja az adott dátumhoz tartozó hétfő 00:00:00 és vasárnap 23:59:59 értékeit. Ezzel kiküszöbölésre került a `toISOString()` miatti UTC időzóna-elcsúszás, ami pontatlan statisztikákat eredményezett volna.

6.3. DNS feloldás kényszerítése (MongoDB Atlas kapcsolat)

„A backend szerver konfigurációjában a Node.js beépített dns moduljának segítségével felülírtuk a hálózat alapértelmezett névfeloldó szervereit a Cloudflare (1.1.1.1) és a Google (8.8.8.8) megbízható, nyilvános szervereire. Ezt a technikai lépést az indokolta, hogy a MongoDB Atlas felhős adatbázis modern kapcsolatfelvételi címeit (SRV rekordok) a lokális internetszolgáltatók (ISP) rendszerei gyakran lassan vagy hibásan oldják fel. Ezzel a két sornyi kóddal a szerver kikerüli a helyi hálózati korlátokat, és garantálja a stabil, azonnali és hibamentes kapcsolatot az adatbázissal.”

```
const dns = require('dns');  
dns.setServers(['1.1.1.1', '8.8.8.8']);
```

5. kód Dns

7. Összegzés, tapasztalatok

A "Kovács Bt. - Munkaóra Nyilvántartó" elkészítése során a Web technológiák 2. tárgy minden lényegi követelményét sikerült gyakorlatba ültetni. A projekt megmutatta a komponens-alapú frontend fejlesztés (Angular) és a RESTful API-k (Node.js) kombinálásának erejét és modularitását.

Fejlesztőként a projekt közben:

Elmélyítettem a tudásomat az Angular Material keretrendszer használatában és a felugró ablakok (Dialogs) egyedi stílusozásában. Megtanultam hatékonyan használni a MongoDB és a Mongoose modelleket az adatok összefűzésére és lekérdezésére. Megtapasztaltam az időzónák és a pontos dátumkezelés kihívásait a JavaScriptben, és sikeresen megoldottam a heti bontású szűréseket. A tovább fejlesztési tervek között szerepel a jelszavak Bcrypt-alapú titkosítása a szerveroldalon, JWT (JSON Web Token) alapú autentikáció bevezetése, valamint az adatok táblázatos (Excel) formátumban történő exportálásának lehetősége az admin felületről.